

Atelier Scikit-learn

Contexte

Une entreprise possède plusieurs bâtiments équipés de capteurs IoT.

Chaque capteur collecte régulièrement des informations sur la température, l'humidité, la pression, la consommation énergétique, le bâtiment, la date et l'heure de la mesure.

Chaque mesure possède également un état (OK, ALERTE et ERREUR).

L'objectif de l'atelier est de construire un modèle capable de **prédire automatiquement l'état d'un capteur à partir de ses mesures**.

L'atelier suivra le workflow classique du Machine Learning :

Dataset → Chargement → Exploration → Nettoyage → X / y → Train / Test → Prétraitement →

Modèle → fit() → predict() → Évaluation → Sauvegarde → Chargement → Réutilisation

Partie 0 – mise en place de l'environnement

1) Structurer le projet comme suit :

```
atelier_scikit-learn_iot/  
├── data/  
│   └── mesures_capteurs.csv  
├── notebooks/  
│   └── atelier_scikit-learn_iot.ipynb  
└── models/  
    ├── modele_capteur.joblib  
    └── modele_capteur.pkl
```

2) Stocker le fichier fourni `mesures_capteurs.csv` dans le sous-dossier `data`

3) Créer le notebook `atelier_scikit-learn_iot.ipynb`

4) Installer et importer `seaborn`, `matplotlib` et `pandas`

5) Importer `mesures_capteurs.csv` dans le dataframe `df`

6) Explorer le dataframe `df`

Partie 1 – Gestion des doublons

Avec Pandas,

1) vérifier l'existence de doublons dans `df`

2) le cas échéant, supprimer les doublons puis vérifier la suppression

Partie 2 – Sélection de y (cible) et X (caractéristiques)

1) Définir `"etat"` comme la cible ou valeur à prédire et `"temperature"`, `"humidite"`, `"pression"` et `"consommation"` comme caractéristiques ou variables explicatives

2) Afficher les cinq premières lignes de `x` et de `y`

3) Quel est le type du problème de machine learning ?

Partie 3 – Découpage Train/Test

Diviser `x` en deux ensembles distincts : un pour l'entraînement (`train`) et un pour le test (`test`).

Avec les conditions suivantes : 20% des données serviront au test ; garantir la reproductibilité du découpage ; conserver les mêmes proportions de classes dans l'ensemble de train et de test que dans les données d'origine.

Partie 4 – Gestion des valeurs manquantes

- 1) Vérifier l'existence de valeurs manquantes
- 2) Sélectionner **SimpleImputer** avec la médiane
- 3) Qu'est ce qui justifie le choix de la médiane ?
- 4) Trouver les paramètres (médianes) de l'imputeur sur **X_train**
- 5) Déterminer **X_train_imputed** et **X_test_imputed**, les transformés de **X_train** et **X_test**

Partie 5 – Mise à l'échelle

- 1) Sélectionner **StandardScaler** pour mettre à l'échelle les transformés de l'imputation
- 2) Qu'est ce qui justifie la standardisation ?
- 3) Trouver les paramètres (moyennes et écart-types) du scaleur sur **X_train_imputed**
- 4) Déterminer **X_train_scaled** et **X_test_scaled**, les transformés de **X_train_imputed** et **X_test_imputed**

Partie 6 – Entrainement et prédiction d'un modèle

- 1) Sélectionner le modèle **KNN** (**k** plus proches voisins) avec 5, le nombre de voisins à prendre en compte
- 2) Entraîner le modèle
- 3) Déterminer **y_pred**, la prédiction du modèle avec l'ensemble de test
- 4) Afficher quelques prédictions
- 5) Comparer ces prédictions avec les vraies valeurs

Partie 7 – Evaluation du modèle

- 1) Evaluer le modèle avec **Accuracy** puis afficher le résultat
- 2) Pourquoi **Accuracy** ne suffit pas toujours pour évaluer ?
- 3) Déterminer, afficher et commenter la matrice de confusion
- 4) Visualiser cette matrice avec **Seaborn**
- 5) Afficher et commenter le rapport de classification
- 6) Dans quel cas de problème de classification est-il plus indiqué d'évaluer avec la **precision** ; même question avec **recall**; même question avec **F1 score**; même question avec **accuracy**

Partie 8 – Sauvegarde du modèle

- 1) Sauvegarder le modèle avec **Joblib** pour avoir **models/modele_capteurs.joblib**
- 2) Sauvegarder le modèle avec **Pickle** pour avoir **models/modele_capteurs.pkl**

Partie 9 – Chargement du modèle

- 1) Charger le modèle
- 2) Faire une prédiction avec une nouvelle mesure

Partie 10 : Bonus

Proposer et réaliser toute fonctionnalité pertinente et utile qu'on pourrait rajouter à l'atelier.

Livrable attendu

Chaque apprenant devra fournir le dossier **atelier_scikit-learn_iot**

Modalité de livraison

Chaque apprenant pousse

- 1) le livrable, **d'abord vide**, sur un dépôt public de son compte Github personnel
- 2) met à jour le livrable, au fur et à mesure, avec des messages de commit explicites